

Relatório - Projeto Final

1. Visão Geral do Sistema

O Projeto Final consolida e expande os desafios anteriores de processamento de texto, unificando-os num único sistema modular e robusto. O programa atua como um motor multifunções capaz de: realizar buscas exatas de padrões (KMP), verificar a conformidade ortográfica de ficheiros (Árvore Trie), sugerir correções inteligentes (Distância de Levenshtein), criptografar e quebrar mensagens secretas (Cifra de César com Busca Binária) e realizar comparações de desempenho de buscas múltiplas (Aho-Corasick).

O sistema foi desenvolvido sob uma rigorosa restrição pedagógica: **nenhuma estrutura de dados complexa ou algoritmo de ordenação/busca nativo da biblioteca padrão (STL) foi utilizado** para a lógica central, garantindo a implementação manual de toda a matemática e a alocação de memória.

2. Arquitetura e Decisões de Design

O projeto adotou o padrão de arquitetura *Controller* (MVC simplificado), abandonando ficheiros únicos gigantescos em prol de uma organização profissional:

- **include/**: Contém os cabeçalhos (.hpp) com as assinaturas e contratos das classes.
- **src/**: Contém as implementações lógicas (.cpp) de cada algoritmo.
- **O Controlador (Main.cpp e MenuMetodos.cpp)**: O ficheiro principal **Main.cpp** foi limpo de qualquer lógica de negócio, contendo apenas um menu iterativo (**switch/case**). As ações de leitura de ficheiros e chamadas de classes foram encapsuladas no **MenuMetodos**, tornando o código escalável.

3. Análise dos Módulos e Algoritmos

Módulo 1: Busca Exata (Knuth-Morris-Pratt - KMP)

Herdado do Projeto 2, este módulo varre um texto em busca de ocorrências exatas de uma palavra.

- **Função **calculaLPS****: Analisa o padrão de busca e cria um vetor (LPS) que armazena o tamanho do maior prefixo que também é sufixo.
- **Como funciona**: Durante a varredura, se houver uma falha de correspondência, o algoritmo consulta o vetor LPS para saber quantos caracteres pode avançar sem precisar de voltar ao início da palavra, garantindo uma complexidade linear $O(N+M)$.

Módulo 2: Verificação Ortográfica (Árvore Trie)

Herdado do Projeto 3, atua como um validador de palavras ultrarrápido.

- **A Estrutura **TrieNode****: Cada nó possui um array de 26 posições (letras de 'a' a 'z') e um marcador booleano **fimDePalavra**.
- **Como funciona**: O ficheiro **words.txt** é carregado no arranque. O sistema possui um **parser (limparPalavra)** que remove pontuações, converte letras para minúsculas e traduz caracteres acentuados UTF-8 (ex: 'á', 'ç') para ASCII básico. A busca de uma

palavra no texto ocorre num tempo proporcional apenas ao tamanho da própria palavra, independentemente do tamanho do dicionário.

Módulo 3: Sugestão de Correção (Distância de Levenshtein)

Quando a Trie deteta que uma palavra não existe, este novo módulo entra em ação para sugerir correções.

- **A Matemática:** Implementa uma matriz de Programação Dinâmica bidimensional. A matriz calcula o custo mínimo de edições (inserção, remoção ou substituição de caracteres) necessárias para transformar a palavra errada numa palavra válida do dicionário.
- **Como funciona:** A função `sugerir` calcula a distância da palavra errada contra todo o dicionário. Para evitar problemas de desempenho e não usar funções prontas, foi implementada uma lógica de ordenação manual (semelhante ao *Bubble Sort* em miniatura) que mantém apenas as 3 melhores sugestões em memória durante o processo.

Módulo 4: Criptografia e Quebra de Cifra

Um simulador de interceptação de mensagens. O sistema cifra uma mensagem e depois utiliza inteligência para a decifrar sem conhecer a chave.

- **Preparação (MergeSort):** Para o ataque funcionar rapidamente, o dicionário precisa de estar ordenado. Foi implementado o algoritmo *MergeSort* do zero (`ordenarDicionario`), que divide o array recursivamente e o conquista, garantindo ordenação em $O(N \log N)$.
- **Força Bruta e Busca Binária:** O algoritmo `quebrarCifra` testa todas as 25 rotações possíveis da Cifra de César. Para cada rotação, o texto gerado é dividido em palavras e validado através de uma **Busca Binária** manual (`buscaBinaria`) no dicionário previamente ordenado. A chave que gerar o maior número de palavras válidas em português é declarada a vencedora.

Módulo 5: Buscador Paralelo e Desempenho (Aho-Corasick)

O desafio final de desempenho para procurar dezenas de palavras ao mesmo tempo.

- **A Estrutura:** Baseia-se na árvore Trie, mas adiciona *Links de Falha* (`fail links`) a cada nó (`ACNode`).
- **Como funciona:** A função `construirAhoCorasick` utiliza uma Busca em Largura (BFS) para calcular para onde o algoritmo deve "saltar" se uma correspondência falhar a meio de uma palavra, sem perder o progresso de outras palavras possíveis.
- **Benchmark:** O sistema realiza uma varredura única no texto e compara o tempo de execução (em microssegundos) do *Aho-Corasick* contra o *KMP* e contra a *Busca Ingênua* (Força Bruta). O Aho-Corasick destaca-se por encontrar múltiplos padrões simultaneamente de forma linear.

4. O Fluxo de Execução do Sistema

1. **Arranque:** O ficheiro compilado (`programa_tbo`) inicia o `Main.cpp`, que apresenta o painel de navegação ao utilizador.

2. **Seleção de Módulo:** O utilizador insere uma opção numérica (0 a 5).
3. **Delegação:** O `Main` invoca a função correspondente no ficheiro `MenuMetodos.cpp`.
4. **I/O e Computação:** O `MenuMetodos` instancia as classes necessárias (ex: `Corretor`, `Cifra`, `Buscador`), trata a abertura dos ficheiros localizados na pasta `Base de dados/` (ex: `dicionario.txt`, `texto_longo.txt`) e imprime os resultados formatados no terminal.
5. **Encerramento de Segurança:** Ao finalizar a execução ou voltar ao menu, os destrutores (`~Dicionario`, `~Buscador`) são acionados, libertando de forma recursiva toda a memória dinâmica alocada pelos ponteiros da árvore e evitando fugas de memória (*memory leaks*).